Les tableaux et les chaînes de caractères

- Les tableaux 1D
- Les algorithmes de tri
- La recherche dichotomique
- Les tableaux 2D
- Les chaînes de caractères

Rado Rakotonarivo (Cours d'Asma Gabis)
rado.rakotonarivo@efrei.fr

P1 – 2024/2025

01

Les tableaux 1D

01

Les tableaux 1D

Idée de base: Rappel

- **Exemple:** 7 variables numériques représentant les notes d'un élève.

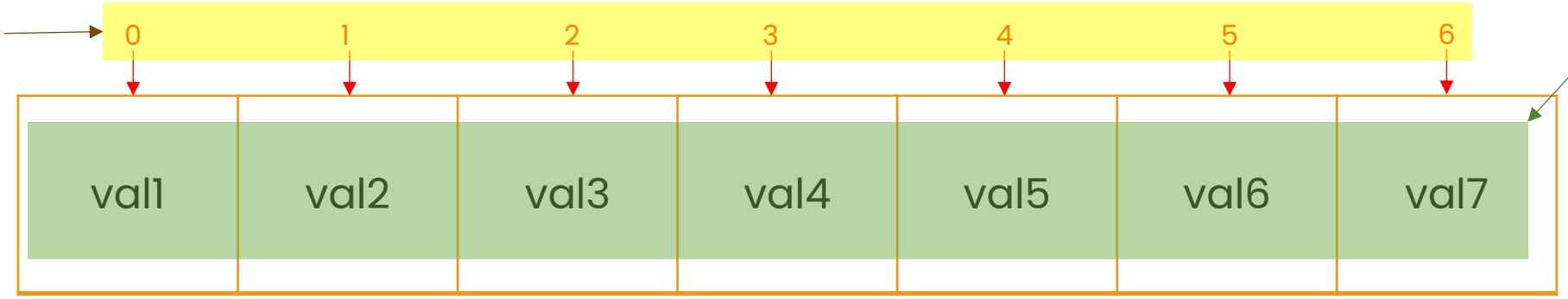
```
int note1;
int note2;
int note3;
int note4;
int note5;
int note6;
int note7;
```

Regrouper les informations dans **un seul et même** contenant appelé : tableau (Array)

Contenu de la case = valeur
Indice (à partir de 0) = position de la valeur

Le nombre de cases (= taille du tableau) correspond au nombre de variables qu'on devait déclarer

A chaque case est associé un indice. Les indices COMMENCENT TOUJOURS A PARTIR DE 0



~~note1 = val1 ;~~
Notes[0] = val1;

~~note4 = val4 ;~~
Notes[3] = val4;

Les tableaux 1D

Taille d'un tableau: Rappel

- Un tableau est caractérisé par :
  - **Sa taille physique** : Nombre de cases réservées
  - **Sa taille logique** : Nombre de cases remplies par des valeurs **significatives**

0	1	2	3	4	5	6	7	8	9	10	11	12	13
-12	11	13	12	23	10	21	15	32	?	?	?	?	?

- Taille physique = 14
- Taille logique = 9
- Les valeurs significatives sont placée dans les cases d'indices 0 ... 8
- Les valeurs en position 9, 10, ... 13 sont indéfinies , donc ignorées

Chaque tableau doit être associé à une variable supplémentaire contenant sa taille logique

Les tableaux 1D

Les tableaux 1D en langage C

- **Déclaration d'un tableau:**

```
Type Nom_tableau[taille_physique];
```

Indique le type de données à stocker dans le tableau

Il n'est pas possible de stocker dans un même tableau des valeurs de types différents

```
int Ar[10]; // tableau pouvant contenir 10 valeurs de type int au max

float myArray[3284]; // tableau pouvant contenir 3284 valeurs de type float au max
```

- **Déclaration de plusieurs tableaux de même type :**

```
Type Tab1[taille_physique1],Tab2[taille_physique2] , …;
```

```
int b[100], x[27];
```

<taille_physique> **DOIT** être une valeur/expression constante

```
#define TAILLE 10

int main()
{
    float averages[15];
    int values[TAILLE]; // <== TAILLE est constante ici
}
```

Les tableaux 1D

Les tableaux 1D en langage C

- **Initialisation d'un tableau:**

```
/* crochets vides et valeurs attribuées entre
accolades
taille physique = taille logique = nombre de
valeurs entre accolades */
int n[] = {1,2,3,4,5};
```

```
/* Si taille physique entre les crochets
 * Si taille physique > nombre de valeurs entre accolades
 * ----- le reste du tableau est initialisé à zéro.
 * Si taille physique < nombre de valeurs entre accolades
 * ----- une erreur de syntaxe est détectée
 * */
int m[5] = {1,2,3,4,5};
int k[5] = {0} //(tous les éléments sont initialisés à zéro)
```

Les tableaux 1D

Les tableaux 1D en langage C

- **Accès aux éléments d'un tableau :**

On accède à un élément d'un tableau en utilisant le nom du tableau et la position de cet élément entre crochets.

```
Nom_tableau[indice_element]
```

Exemple :

```c
#include <stdio.h>

int main()
{
    int premiers[] = {2, 3, 5, 7, 11, 13, 17, 19};
    // À noter que c'est la 4ème valeur de la liste
    printf("La valeur à l'indice 3 est : %d.\n", premiers[3]);

    // Car la numérotation commence à partir de 0
    printf("La 5ème valeur de cette liste est : %d.\n", premiers[4]);
}
```

```
La valeur à l'indice 3 est : 7.
La 5ème valeur de cette liste est : 11.
```

Les tableaux 1D

Les tableaux 1D en langage C

- **Remplir un tableau :**

Deux étapes :

1. Déterminer la taille logique (fixe ou saisie par l'utilisateur) <= taille physique
2. Remplir les éléments d'un tableau un par un ➔ utiliser boucle

Exemple :

```c
#include <stdio.h>

int main()
{
    int ar[20];
    int Taille_logique = 10;

    // Saisir 10 entiers
    printf("Saisir %d entiers: \n", Taille_logique);
    // Boucle pour saisir les valeurs une à une
    for (int i = 0; i < Taille_logique; i++)
    {
        scanf("%d", &ar[i]);
    }
}
```

Une case est saisie comme une variable

Les tableaux 1D

Les tableaux 1D en langage C

- **Affichage des éléments d'un tableau :**

  Les éléments d'un tableau doivent s'afficher un par un .

Exemple :

```
#include <stdio.h>

int main()
{
    int ar[] = {2, 12, 34, 11, 8};
    int Taille_logique = 5;

    // Affichage des éléments d'un tableau
    for (int i = 0; i < Taille_logique; i++)
    {
        printf("%d ", ar[i]); // séparer les valeurs par des espaces
    }
    printf("\n");
}
```

Les tableaux 1D

Erreurs à éviter (confusion avec Python)

- **Taille physique non constante :**

  ```
  int n; ... int T[n]; int T[2*n];
  ```

  À ne pas faire (même si ça peut passer à la compilation)

- **Saisie d'un tableau**

  ```
  int T[10];
  scanf("%d", T);
  ```

  À ne pas faire, il faut remplir les cases une par une (boucle)

- **Affichage d'un tableau**

  ```
  printf("%d", T);
  ```

  À ne pas faire, il faut afficher les cases une par une (boucle)

Les tableaux 1D

Erreurs à éviter (confusion avec Python)

- **Indice négatif, et taille** :

```
int T[10];
T[-1]=0;
n=len(T);
```

À ne pas faire, on ne peut pas avoir la taille d'un tableau.

- **Comparaison de deux tableaux**

```
int T1[10], T2[10] ...
if(T1==T2)
...
```

A ne pas faire, on doit comparer le contenu case par case.

02

Les algorithmes de tri

02

Trier un tableau 1D

Définition :

- Un algorithme de tri est un algorithme qui permet d'organiser les éléments d'un tableau dans un ordre donné:
  - **Croissant** : de la plus petite valeur à la plus grande
  - **Décroissant** : de la plus grande valeur à la plus petite.

- **Objectif principal** : Faciliter la recherche dans un tableau.
- Plusieurs stratégies de tri existent : Pourquoi ?
  - Recherche de l'algorithme le plus optimal en nombres d'opérations effectuées (comparaisons, permutations, …etc.)
- **Exemples** : Tri à bulles, Tri par insertion, Tri par sélection, Etc.

Trier un tableau 1D

Tri à bulles :

- Un des plus simples algorithmes de tri.
- **Principe** :
  - Parcourir un tableau du début à la fin et comparer les valeurs de deux cases consécutives
  - Si les valeurs ne sont pas rangées dans l'ordre souhaité, alors elles sont permutées.
  - Continuer le processus jusqu'à ce que le tableau soit entièrement trié.
- **Pseudo algorithme** :

```
Boucle 1: Parcours jusqu'à ce que chaque élément soit placé dans sa position définitive.
     Boucle 2 : (j) de 1 à la taille logique - 1.
          Comparaison éléments aux positions j et (j+ 1).
          Permutation si ces deux éléments ne sont pas dans l'ordre désiré.
     Fin Boucle 2
     Décrémenter la taille de la "zone" non triée de 1.
Fin Boucle 1
```

Trier un tableau 1D

Tri à bulles :

- **Programme C** :

bubble_sort.c

```c
#include <stdio.h>

#define NMAX 1000

int main() {
    int t[NMAX];
    int n, i, j, temp;

    // Read the number of elements
    do {
        printf("Enter the number of elements: ");
        scanf("%d", &n);
    } while (n <= 0 || n > NMAX);

    // Read the elements
    printf("Type the %d elements:\n", n);
    for (i = 0; i < n; i++) {
        printf("t[%d] = ", i);
        scanf("%d", &t[i]);
    }

    // Sort t using bubble sort
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (t[j] > t[j + 1]) {
                temp = t[j];
                t[j] = t[j + 1];
                t[j + 1] = temp;
            }
        }
    }

    // Print the sorted array
    printf("Sorted array:\n");
    for (i = 0; i < n; i++) {
        printf("%d ", t[i]);
    }
    printf("\n");

    return 0;
}
```

Trier un tableau 1D

Tri à bulles :

Exemple : Appliquer le programme précédent pour trier le tableau ci-dessous dans l'ordre croissant.

13	7	43	5	3	19	2	23	29	?	?	?	?	?

Tableau initial	13	7	43	5	3	19	2	23	29
itération 1	7	13	5	3	19	2	23	29	43
itération 2									
itération 3									
itération 4									
itération 5									
itération 6									
itération 7									
itération 8									

Permutation des deux valeurs pour correspondre à l'ordre croissant souhaité

Trier un tableau 1D

Tri à bulles :

Tableau initial	13	7	43	5	3	19	2	23	29
itération 1	7	13	5	3	19	2	23	29	43
itération 2	7	5	3	13	2	19	23	29	43
itération 3									
itération 4									
itération 5									
itération 6									
itération 7									
itération 8									

À la fin de la deuxième boucle, la valeur maximum est positionnée à la fin

Trier un tableau 1D

Tri à bulles :

Tableau initial	13	7	43	5	3	19	2	23	29
itération 1	7	13	5	3	19	2	23	29	43
itération 2	7	5	3	13	2	19	23	29	43
itération 3	5	3	7	2	13	19	23	29	43
itération 4									
itération 5									
itération 6									
itération 7									
itération 8									

Trier un tableau 1D

Tri à bulles :

Tableau initial	13	7	43	5	3	19	2	23	29
itération 1	7	13	5	3	19	2	23	29	43
itération 2	7	5	3	13	2	19	23	29	43
itération 3	5	3	7	2	13	19	23	29	43
itération 4	3	5	2	7	13	19	23	29	43
itération 5									
itération 6									
itération 7									
itération 8									

Trier un tableau 1D

Tri à bulles :

Tableau initial	13	7	43	5	3	19	2	23	29
itération 1	7	13	5	3	19	2	23	29	43
itération 2	7	5	3	13	2	19	23	29	43
itération 3	5	3	7	2	13	19	23	29	43
itération 4	3	5	2	7	13	19	23	29	43
itération 5	3	2	5	7	13	19	23	29	43
itération 6									
itération 7									
itération 8									

Trier un tableau 1D

Tri à bulles :

Tableau initial	13	7	43	5	3	19	2	23	29
itération 1	7	13	5	3	19	2	23	29	43
itération 2	7	5	3	13	2	19	23	29	43
itération 3	5	3	7	2	13	19	23	29	43
itération 4	3	5	2	7	13	19	23	29	43
itération 5	3	2	5	7	13	19	23	29	43
itération 6	2	3	5	7	13	19	23	29	43
itération 7	2	3	5	7	13	19	23	29	43
itération 8	2	3	5	7	13	19	23	29	43

L'algorithme continue à tourner et vérifier les valeurs tant qu'ils ne les a pas toutes parcouru , pourtant tableau trié à la 6ème itération ➔

Problème d'efficacité !!!!

Comment l'améliorer ?

Trier un tableau 1D

Tri par sélection :

- Un autre algorithme de tri.
- **Principe** : Pour un tri dans l'ordre croissant
  - Rechercher la valeur minimum (récupérer sa position)
  - La permuter avec la première valeur du tableau ➔ Le minimum occupe sa position finale.
  - Considérer la zone non triée et reproduire les deux étapes précédentes.
- **Pseudo algorithme**

```
  Boucle 1 (i) de 1 au nombre de valeurs à trier - 1.
     Hypothèse le minimum courant se trouve à la position i.
     Boucle 2 (j) parcourir le reste du tableau à trier (i+1 ... size-1).
           Comparer la Jème valeur au minimum courant.
           Si la Jème valeur est < au minimum courant alors
                   le minimum courant prend la valeur à la Jème position.
           Fin Si
      Fin Boucle 2
      Déplacer le minimum au début de la partie non triée du tableau (c.à.d. permuter la ième valeur et le
minimum).
  Fin Boucle 1
```

Trier un tableau 1D

Tri par sélection :

- **Programme C**

```c
#include <stdio.h>

#define TAILLE 10
int main()
{
    int A[TAILLE] ;
    int Taille_logique, i ,j, temp, min;

    // Saisie de la taille logique
    do
    {
        printf("Saisir la taille logique: \n");
        scanf("%d", &Taille_logique);
    } while (Taille_logique <= 0);

    // Saisie des éléments du tableau
    printf("Saisir %d valeurs : \n", Taille_logique);
    for (i = 0; i < Taille_logique; i++)
    {
        scanf("%d", &A[i]);
    }

    // Trier le tableau par un tri par sélection
    for(i = 0; i < Taille_logique - 1 ; i++)
    {
        min = i;
        for(j = i+1; j < Taille_logique; j++)
        {
            if( A[j] < A[min])
            {
                min = j;
            }
        }

        if ( min != i)
        {
            temp = A[i];
            A[i] = A[min];
            A[min] = temp;
        }
    }
}
```

Recherche de la valeur min dans la zone non triée.

Permutation du min avec la première valeur de la zone non triée.

Trier un tableau 1D

Tri par sélection :

Exemple : Appliquer le programme précédent pour trier le tableau ci-dessous dans l'ordre croissant.

13	7	43	5	3	19	2	23	29	?	?	?	?	?

Tableau initial	13	7	43	5	3	19	2	23	29
itération 1	2	7	43	5	3	19	13	23	29
itération 2									
itération 3									
itération 4									
itération 5									
itération 6									
itération 7									
itération 8									

Valeur minimum du tableau

Permuter la valeur min avec la première valeur du tableau

Trier un tableau 1D

Tri par sélection :

Zone non encore triée

ableau initial	13	7	43	5	3	19	2	23	29
itération 1	2	7	43	5	3	19	13	23	29
itération 2	2	3	43	5	7	19	13	23	29
itération 3									
itération 4									
itération 5									
itération 6									
itération 7									

Valeur minimum de la partie du tableau non triée.

Déjà à sa position finale

Permuter la valeur min avec la première valeur de la partie non triée du tableau

Trier un tableau 1D

Tri par sélection :

Après plusieurs itérations …

Tableau initial	13	7	43	5	3	19	2	23	29
itération 1	2	7	43	5	3	19	13	23	29
itération 2	2	3	43	5	7	19	13	23	29
itération 3	2	3	5	43	7	19	13	23	29
itération 4	2	3	5	7	43	19	13	23	29
itération 5	2	3	5	7	13	19	43	23	29
itération 6	2	3	5	7	13	19	43	23	29
itération 7	2	3	5	7	13	19	23	43	29
itération 8	2	3	5	7	13	19	23	29	43

La recherche dichotomique

Principe :

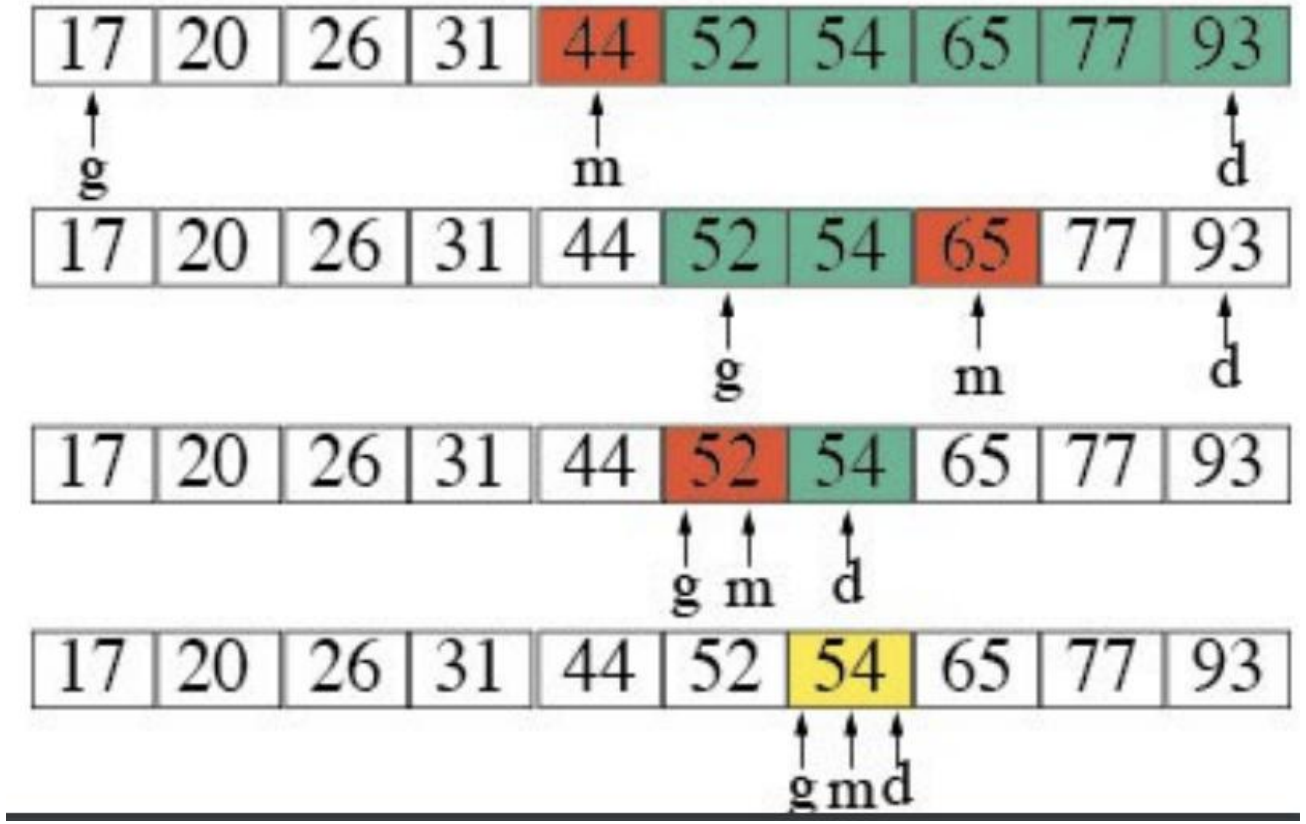
Appliquée sur un tableau trié

- **Algorithme :** A chaque étape :
  - Découpage du tableau ***T*** en deux sous-tableau à l'aide d'un indice médian (moitié inférieure) et (moitié supérieure)
  - Utiliser 2 indices gauche (***g***) et droite (***d***)
  - Comparaison de la valeur située à l'indice central (m) à la valeur recherchée,
    1. Si la valeur recherchée est supérieure ou égal à la valeur ***T[m]*** relancer la recherche avec la moitié supérieure,
    2. Sinon relancer la recherche avec la moitié inférieure.

La recherche dichotomique

Exemple :

- valeur recherchée = 54



La recherche dichotomique

- **Programme C**

binary_search.c

```c
#include <stdio.h>

#define N 10

int main () {
    // Init an sorted array
    int t[N] = {14, 20, 26, 31, 44, 53, 54, 65, 77, 93};
    // Read the value to search
    int value = 0;
    scanf("%d", &value);
    // Initialize program's variables
    int l = 0, r = N - 1;
    int index = -1;
    int found = 0;
    // Binary search
    while (l <= r && !found) {
        int m = r - (r - l) / 2; // Get the mid index
        if (t[m] == value) { // Value found. Wa happy :)
            found = 1;
            index = m;
        } else if (t[m] < value) {
            l = m + 1; // Search in the left subarray
        } else {
            r = m - 1; // Search in the right subarray
        }
    }

    if (found) {
        printf("Value found at index %d\n", index);
    } else {
        printf("Value not found\n");
    }

    return 0;
}
```

Dans le programme,
- l correspond à g (pour left)
- r correspond à d (pour right)
- m correspond à m (pour middle)

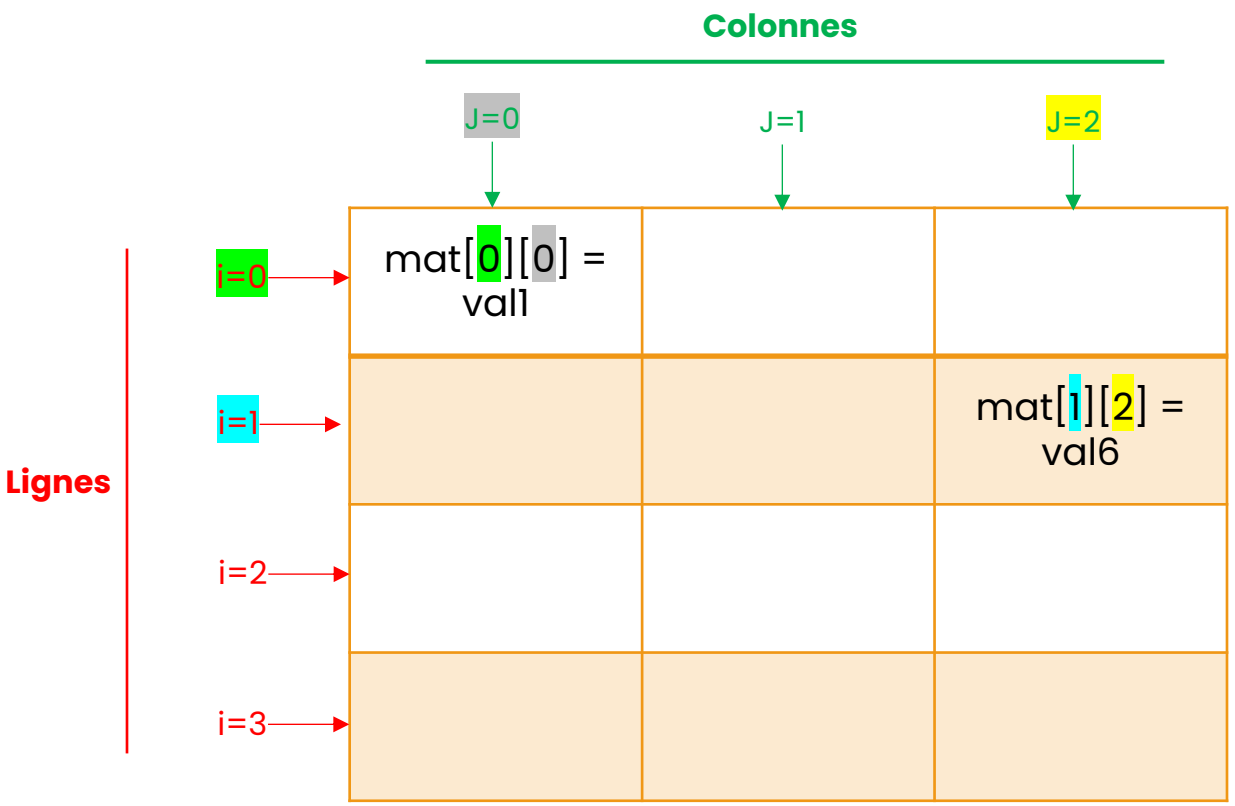
03

Les tableaux 2D

03

Les tableaux 2D

Représentation générale : Rappel



Les tableaux 2D

Manipulation d'un tableau 2D

- **Déclaration d'un tableau 2D :**

Type Nom_Mat[TP_lig] [TP_col];

Nombre de lignes maximum réservées

Nombre de colonnes maximum réservées

```
#include <stdio.h>

#define NB_LIG 20
#define NB_COL 10

int main()
{
    int M1[100][100];
    int M2[NB_LIG][NB_COL];
    return 0;
}
```

- **Initialisation d'un tableau 2D :**

```
int a[3][4] = {
{0,1,2,3} , /*ligne d'indice 0 */
{4,5,6,7} , /*ligne d'indice 1 */
{8,9,10,11} /*ligne d'indice 2 */
};
```

- **Accès à une case d'un tableau 2D :**

Pour accéder à une case, deux indices sont nécessaires.

```
scanf("%d", &M1[0][0]); // Saisie de la case d'indices (0, 0)

printf("%d", M2[0][1]); // Affichage de la case d'indices (0, 1)
```

Les tableaux 2D

Manipulation d'un tableau 2D

- **Remplir un tableau 2D :**

```c
#include <stdio.h>

#define NB_LIG 20
#define NB_COL 10

int main()
{
    int M1[NB_LIG][NB_COL];
    int i, j, lig = 3, col = 4;

    for (i = 0; i < lig; i++)
    {
        for (j = 0; j < col; j++)
        {
            printf("Saisir une valeur : ");
            scanf("%d", &M1[i][j]);
        }
    }
    return 0;
}
```

- **Affichage d'un tableau 2D :**

```c
#include <stdio.h>

#define NB_LIG 20
#define NB_COL 10

int main()
{
    // initialisation de la matrice en séquence
    int M[3][4] = {0,1,2,3,4,5,6,7,8,9,10,11};
    int i, j, lig = 3, col = 4;

    for (i = 0; i < lig; i++)
    {
        for (j = 0; j < col; j++)
        {
            printf("%d \t\t", M[i][j]); // séparer les valeurs
d'une même ligne par 2 tabulations
        }
        printf("\n"); // Saut de ligne avant d'afficher la suivante
    }
    return 0;
}
```

04

Les chaînes de caractères

04

Les chaînes de caractères

Définition

Une chaîne de caractères est définie comme étant un tableau à 1D de caractères qui se termine par le caractère **'\0'** (Visuellement composé de deux caractères mais ne représentant qu'un seul).

Déclaration :

Caractère Nom_chaine[taille_physique];

Tableau de caractères

```
// Tableau de caractères + '\0' = chaîne de caractères
// Taille tableau = nombre de caractères + 1 ('\0')

char greeting[6] = {'H', 'E', 'L', 'L', 'O', '\0'};
```

```
// affectation d'une chaine de caractères
// Le compilateur C place automatiquement le caractères
'\0'
char salutation[] = "Hello";
```

Les chaînes de caractères

Représentation en mémoire

```
char salutation[] = "Hello";
```

- L'accès à chacun des caractères se fait par l'indice.
- Les indices commencent à 0.

Index	0	1	2	3	4	5
Variable	H	e	l	l	o	\0
Address	0x23451	0x23452	0x23453	0x23454	0x23455	0x23456

Les chaînes de caractères

Manipulation d'une chaîne de caractères

- **Affichage d'une chaîne de caractères:**

Elle s'affiche en bloc avec le format **%s**

```
char greeting[6] = {'H', 'E', 'L', 'L', 'O', '\0'};

char salutation[] = "Hello";

printf("%s", salutation);
printf("%s", greeting);
```

- **Saisie d'une chaîne de caractères:**

- La saisie se fait en bloc avec le format **%s**
- Pas de **'&'** pour la saisie d'une chaîne de caractères

```
char My_str[32];
scanf("%s", My_str);
```

- **Accéder à un caractère dans une chaîne de caractères:**

L'accès se fait directement par l'indice.

```
printf("%c", salutation[0]);
```

Les chaînes de caractères

Manipulation d'une chaîne de caractères

- **Saisie d'une phrase:**

  Une phrase = une suite de mots séparés par des espaces

  Effet de l'utilisation du scanf

```
char My_str[32];
printf("Saisir une chaine de caractères : \n");
scanf("%s", My_str);
printf("%s \n", My_str);
```

```
Saisir une chaine de caractères :
Hello World !
Hello
```

La chaîne stockées est incomplète.
scanf s'arrête au premier espace rencontré.

- **Solution : Utilisation de gets**

  Avec **gets**, la saisie s'arrête au retour à la ligne **'\n'**

  gets ne prend pas de format en argument.

```
char My_str[32];
printf("Saisir une chaine de caractères : \n");
gets(My_str);
printf("%s \n", My_str);
```

```
Saisir une chaine de caractères :
Hello World !
Hello World !
```

Tableaux VS Chaînes de caractères

	Tableau	Chaîne de caractères
Affichage	Case par case → boucle	En bloc
Saisie	Case par case → boucle	En bloc
Scanf	Formats %d, %f, %c, … nécessitant un '&'	Format %s ne nécessitant pas de '&'
Condition d'arrêt	Taille logique	Le caractère '\0'
Accès aux éléments	**Par indice**	**Par indice**